МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 6

Варіант 6

Замикання та декоратори як HOF

Виконав: студент 2 курсу групи ФЕП-22
Мазан Давид

Перевірив: аспірант
Остріков Данило Андрійович

Львів - 2026

Мета роботи

Навчитися створювати замикання зі станом, параметризовані декоратори й функції вищого порядку. Реалізувати таймінг, повторні спроби та кешування з інвалідацією. Для варіанта 6 симулювати файлову операцію з тимчасовою помилкою та успішним повторним викликом.

Теоретичні відомості

Замикання зберігає доступ до значень зовнішньої області видимості, а `nonlocal` дозволяє змінювати такий стан. Параметризований декоратор зберігає налаштування у closure і повертає обгортку функції. functools.wraps зберігає ім'я, документацію та __wrapped__; ParamSpec/TypeVar описують типи аргументів і результату.

Ін'єкція годинника, sleep, випадкового значення й logger робить декоратори детермінованими для тестів. TTL-кеш перевіряє строк дії при наступному зверненні та підтримує повне чи вибіркове очищення.

Постановка завдання

Варіант 6: симулювати тимчасові помилки читання/запису та повторювати операцію до успіху. Додати closure-логер, декоратор повторів і статистику спроб. Загальні вимоги: два замикання, зокрема стан через nonlocal; параметризовані декоратори з wraps, ParamSpec/TypeVar та ін'єкцією залежностей; кешування з TTL та інвалідацією.

Хід роботи

1. У closures.py створено make_counter зі змінним станом, make_logger з ін'єкцією sink і closure-backed статистику повторів.

2. У decorators.py реалізовано timed, retries, cached і retry_statistics. Декоратор retries підтримує типи винятків, backoff, jitter та ін'єктовані sleep/logger/random_fn/on_attempt. Кеш підтримує TTL, cache_clear і вибіркову інвалідацію.

3. У app.py файлова операція симулюється функцією: перші два виклики кидають OSError, наступний повертає результат. Годинник і затримки підставні, справжній I/O і sleep не використовуються.

4. Тести перевіряють стан closures, таймінг при успіху й винятку, повтори, backoff, TTL, інвалідацію, статистику та метадані функцій.

Структура проєкту

```
lab06/
├── report/ (lab6.docx, lab6.pdf)
├── tests/ (test_closures, test_timed, test_retries, test_cached, test_composition)
├── closures.py
├── decorators.py
├── app.py
├── ASSIGNMENT.md
├── README.md
├── requirements.txt
├── mypy.ini
└── pyproject.toml
```

Результати виконання

Симульований виклик успішно завершився після трьох спроб. Підставний sleep зафіксував затримки 0.01 і 0.02 секунд, а injected clock — 5.000 ms. Кешований результат повернувся повторно без обчислення; після точкової інвалідації функція обчислилася ще раз.

```
=== Замикання та декоратори: варіант 6 ===
Результат операції: contents of report.txt
Затримки між повторами: [0.01, 0.02]
Статистика: attempts=3, calls=1, successes=1, failures=0
Кешовані розміри: 100, 100, 100
Видалено з кешу: 1; обчислень: 2
Таймінг: 5.000 ms
```

Команду pytest -q --durations=5 виконано: 15 passed in 0.08s. Муру завершився без помилок для трьох модулів; ruff check пройшов; усі Python-файли перевірені black.

Висновок

Реалізовано замикання зі станом, логер, статистику повторів і параметризовані декоратори таймінгу, повторів та кешування. Ін’єкція залежностей дозволяє тестувати часові сценарії без затримок. TTL і вибіркова інвалідація працюють за ключами аргументів. Усі 15 тестів пройшли.